

Análisis de Patrones de Diseño y Arquetipos Maven

Donaton — Evaluación Parcial 2

DSY1106 Desarrollo Fullstack III | DuocUC

Integrantes	Oscar Flores · Leroy Rodriguez
Profesor	Juan Valentin Mora Ruiz
Fecha	Mayo 2026
Componentes	MS-Donaciones (8081) · MS-Logística (8082) · BFF (8080) · Frontend React

1. Arquitectura General

La solución Parcial 2 extiende la arquitectura de microservicios del Parcial 1 con implementaciones concretas separadas en tres componentes backend independientes y un frontend React, conectados mediante el patrón Backend For Frontend.

Componente	Puerto	Patrón(es) Principal(es)
MS-Donaciones	8081	Repository Pattern + Factory Method
MS-Logística	8082	Repository Pattern + Observer
BFF-Donaton	8080	Backend For Frontend (BFF)
Frontend React	3000	Facade + Custom Hook

2. Patrones de Diseño Implementados

2.1 Repository Pattern (Estructural)

Dónde se aplica: MS-Donaciones y MS-Logística

Problema que resuelve: En el código original del Parcial 1, los servicios accedían directamente a listas en memoria, acoplando la lógica de negocio a la implementación de persistencia. Cualquier cambio a PostgreSQL requeriría modificar todos los servicios.

Implementación:

```
// Interfaz – contrato de acceso a datos
public interface DonacionRepository {
    List<Donacion>    findAll();
    Optional<Donacion> findById(Long id);
    List<Donacion>    findByEstado(String estado);
    Donacion          save(Donacion donacion);
    boolean            deleteById(Long id);
}
```

```

// Implementación en memoria (intercambiable por JPA sin tocar el servicio)
@Repository
public class DonacionRepositoryImpl implements DonacionRepository { ... }

// Servicio – solo conoce la interfaz
@Service
public class DonacionService {
    private final DonacionRepository donacionRepository; // inyectado
}

```

- La lógica de negocio nunca accede directamente a la base de datos.
- Cambiar de memoria a PostgreSQL no requiere modificar DonacionService ni los controllers.
- Las pruebas unitarias usan mocks del repositorio para aislar el servicio completamente.

2.2 Factory Method (GoF — Creacional)

Dónde se aplica: MS-Donaciones — paquete factory/

Problema que resuelve: Donaton maneja cuatro tipos de donación (ROPA, ALIMENTO, MEDICO, HIGIENE), cada uno con sus propias reglas de validación y estado inicial. Sin Factory Method, DonacionService tendría un bloque if/switch que viola el principio Open/Closed.

Implementación:

```

// Clase abstracta – Template Method + Factory Method
public abstract class DonacionFactory {
    public final Donacion crear(String origen, Integer cantidad, ...) {
        validarParametros(origen, cantidad); // validación común
        Donacion d = construir(...); // delegado a subclase
        aplicarReglasTipo(d); // reglas específicas del tipo
        return d;
    }
    protected abstract Donacion construir(...);
    protected abstract void aplicarReglasTipo(Donacion d);
}

// Fábrica concreta – insumos médicos tienen prioridad alta
@Component
class DonacionMedicoFactory extends DonacionFactory {
    protected void aplicarReglasTipo(Donacion d) {
        d.setEstado("EN_PROCESO"); // regla: insumos médicos pasan directo
    }
}

// Registro dinámico: Spring inyecta todas las fábricas automáticamente
@Component
public class DonacionFactoryProvider {
    private final Map<String, DonacionFactory> fabricas;
    public DonacionFactory obtenerFabrica(String tipo) { ... }
}

```

- Agregar un nuevo tipo de donación (ej: TECNOLOGÍA) solo requiere una nueva clase sin modificar código existente.
- Cada tipo encapsula sus propias reglas de validación y estado inicial (Open/Closed Principle).
- El DonacionFactoryProvider actúa como registro dinámico inyectado por Spring.

2.3 Observer (GoF — Comportamiento)

Dónde se aplica: MS-Logística — paquete observer/

Problema que resuelve: Cuando un envío cambia de estado (PLANIFICADO → EN_CAMINO → ENTREGADO), el sistema debe registrar en auditoría y enviar notificaciones. Sin Observer, LogisticaService mezclaría auditoría, notificaciones y métricas, violando el Single Responsibility Principle.

Implementación:

```
// Interfaz del observador
public interface EnvioObserver {
    void onEnvioActualizado(Envio envio, String estadoAnterior);
}

// Observador 1: Auditoría
@Component
public class AuditoriaEnvioObserver implements EnvioObserver {
    public void onEnvioActualizado(Envio envio, String estadoAnterior) {
        bitacora.add(String.format("Envío #%d: %s → %s", envio.getId(),
            estadoAnterior, envio.getEstado()));
    }
}

// Observador 2: Notificaciones
@Component
public class NotificacionEnvioObserver implements EnvioObserver {
    public void onEnvioActualizado(Envio envio, String estadoAnterior) {
        // Dispara SMS/email según el nuevo estado
    }
}

// Servicio (sujeto): notifica a todos sin conocerlos explícitamente
@Service
public class LogisticaService {
    private final List<EnvioObserver> observadores; // Spring inyecta todos
    private void notificarObservadores(Envio envio, String estadoAnterior) {
        for (EnvioObserver obs : observadores) obs.onEnvioActualizado(envio, estadoAnterior);
    }
}
```

- Nuevos comportamientos (métricas, webhooks) se agregan sin modificar LogisticaService.
- Cada observador tiene una única responsabilidad (Single Responsibility Principle).
- Spring gestiona automáticamente el registro de observadores mediante inyección de dependencias.

2.4 Backend For Frontend (Arquitectónico)

Dónde se aplica: bff-donaton (puerto 8080)

Problema que resuelve: El frontend necesitaría 3 llamadas HTTP separadas para construir el dashboard, generando latencia acumulada y lógica de composición en el cliente React.

```
// El BFF agrega 3 fuentes en 1 respuesta
@GetMapping("/bff/dashboard")
public DashboardResumenDTO getDashboard() {
    // 1 llamada del frontend → 3 llamadas internas del BFF
}
```

```

        return bffService.obtenerResumenDashboard();
    }

    // Si MS-Logística falla, el BFF devuelve datos de Donaciones + alerta
    // (resiliencia parcial sin lanzar excepción)

```

- El frontend realiza 1 llamada en lugar de 3 (reducción de latencia percibida).
- El BFF maneja fallos parciales: si un microservicio no responde, devuelve los demás datos con alerta.
- El payload está optimizado para la vista específica del cliente (DashboardResumenDTO).

3. Arquetipos Maven

Se implementó el arquetipo **donaton-microservice-archetype** para estandarizar la creación de nuevos microservicios. Este arquetipo genera automáticamente la estructura base con Repository Pattern preconfigurado, REST Controller con CRUD completo, pruebas unitarias con Mockito y reporte de cobertura JaCoCo.

3.1 Uso del Arquetipo

```

# Instalar arquetipo en repositorio local
cd arquetipos-maven/donaton-microservice-archetype && mvn install

# Generar nuevo microservicio
mvn archetype:generate \
  -DarchetypeGroupId=donaton.arquetipos \
  -DarchetypeArtifactId=donaton-microservice-archetype \
  -DarchetypeVersion=1.0.0 \
  -DgroupId=donaton \
  -DartifactId=ms-voluntarios \
  -DservicePuerto=8083

```

3.2 Estructura Generada

Archivo generado	Descripción
pom.xml	Dependencias Spring Boot 3.x + JaCoCo
Application.java	Clase principal Spring Boot
model/Entidad.java	Modelo de dominio base (renombrar al dominio real)
repository/EntidadRepository.java	Interfaz Repository Pattern
repository/EntidadRepositoryImpl.java	Implementación en memoria
service/EntidadService.java	Servicio de negocio con inyección del repositorio
controller/EntidadController.java	REST Controller CRUD completo
service/EntidadServiceTest.java	Pruebas unitarias con Mockito preconfiguradas
application.properties	Puerto parametrizado con \${servicePuerto}

Justificación: Sin el arquetipo, cada nuevo microservicio requiere replicar manualmente la estructura base, introduciendo inconsistencias. El arquetipo garantiza que todos los microservicios implementen el Repository Pattern y cuenten con pruebas unitarias desde el inicio.

4. Resumen de Patrones por Componente

Componente	Patrón	Clasificación	Beneficio Principal
MS-Donaciones	Repository Pattern	Estructural	Desacoplamiento de persistencia
MS-Donaciones	Factory Method	Creacional (GoF)	Open/Closed Principle
MS-Logística	Repository Pattern	Estructural	Desacoplamiento de persistencia
MS-Logística	Observer	Comportamiento (GoF)	Single Responsibility
BFF-Donaton	Backend For Frontend	Arquitectónico	Reducción de llamadas HTTP
Frontend React	Facade	Estructural	Encapsulamiento HTTP
Frontend React	Custom Hook	Comportamiento	Separación de responsabilidades